Chapter 20. Blending OOP and FP: Comparing Java and Scala

This chapter covers

- An introduction to Scala
- How Java relates to Scala and vice versa
- How functions in Scala compare to Java
- Classes and traits

Scala is a programming language that mixes object-oriented and functional programming. It's often seen as an alternative language to Java for programmers who want functional features in a statically typed programming language that runs on the JVM while keeping a Java feel. Scala introduces many more features than Java: a more-sophisticated type system, type inference, pattern matching (as presented in **chapter 19**), constructs that define domain-specific languages simply, and so on. In addition, you can access all Java libraries within Scala code.

You may wonder why we have a chapter about Scala in a Java book. This book has largely centered on adopting functional-style programming in Java. Scala, like Java, supports the concepts of functional-style processing of collections (that is, streamlike operations), first-class functions, and default methods. But Scala pushes these ideas further, providing a larger set of features that support these ideas compared with Java. We believe that you may find it interesting to compare Scala with the approach taken by Java and see Java's limitations. This chapter aims to shed light on this matter to appease your curiosity. We don't necessarily encourage the adoption of Scala over Java. Other interesting new programming languages on the JVM, such as Kotlin, are also worth looking at. The purpose of this chapter is to open your horizons to what's available beyond Java. We believe that it's important for a well-rounded software engineer to be knowledgeable about the wider programming-languages ecosystem.

Also keep in mind that the purpose of this chapter isn't to teach you how to write idiomatic Scala code or to tell you everything about Scala. Scala pattern matching, for-comprehensions, ble in Java, and we won't discuss those mparing the Java and Scala features to give you an idea of the bigger picture. You'll find that you can write more concise and readable code in Scala compared with Java, for example.

This chapter starts with an introduction to Scala: writing simple programs and working with collections. Next, we discuss functions in Scala: first-class functions, closures, and currying. Finally, we look at classes in Scala and at a feature called traits, which is Scala's take on interfaces and default methods.

20.1. Introduction to Scala

This section briefly introduces basic Scala features to give you a feel for simple Scala programs. We start with a slightly modified "Hello world" example written in an imperative style and a functional style. Then we look at some data structures that Scala supports—`List`, `Set`, `Map`, `Stream`, `Tuple`, and `Option`—and compare them with Java. Finally, we present *traits*, Scala's replacement for Java's interfaces, which also support inheritance of methods at object-instantiation time.

20.1.1. Hello beer

To change a bit from the classic "Hello world" example, bring in some beer. You want to print the following output on the screen:

```
Hello 2 bottles of beer
Hello 3 bottles of beer
Hello 4 bottles of beer
Hello 5 bottles of beer
Hello 6 bottles of beer
```

Imperative-style Scala

Here's how the code to print this output looks in Scala when you use an imperative style:

```
object Beer {
  def main(args: Array[String]){
    var n : Int = 2
    while( n <= 6){
      println(s"Hello ${n} bottles of beer")          1
      n += 1
    }
  }
}
```


You can find information about how to run this code on the official Scala website (see **https://docs.scala-lang.org/getting-started.html**). This program looks similar to what you'd write in Java, and its structure is similar to that of Java programs, consisting of one method called `main`, which

takes an array of strings as argument. (Type annotations follow the syntax `s : String` instead of `Strings`, as in Java.) The `main` method doesn't return a value, so it's not necessary to declare a return type in Scala as you'd have to do in Java when you use `void`.

Note

In general, nonrecursive method declarations in Scala don't need an explicit return type, because Scala can infer the type for you.

Before we look at the body of the `main` method, we need to discuss the `object` declaration. After all, in Java you have to declare the method `main` within a class. The declaration `object` introduces a singleton object, declaring a class `Beer` and instantiating it at the same time. Only one instance is created. This example is the first example of a classical design pattern (the singleton design pattern) implemented as a language feature, and it's free to use out of the box. In addition, you can view methods within an `object` declaration as being declared as static, which is why the signature of the `main` method isn't explicitly declared as `static`.

Now look at the body of `main`. This method also looks similar to a Java method, but statements don't need to end with a semicolon (which is optional). The body consists of a `while` loop, which increments a mutable variable, `n`. For each new value of `n`, you print a string on the screen, using the predefined `println` method. The `println` line showcases another feature of Scala: *string interpolation*, which allows you to embed variables and expressions directly in string literals. In the preceding code, you can use the variable `n` directly in the string literal `s"Hello ${n} bottles of beer"`. Prepending the string with the interpolator `s` provides that magic. Normally in Java, you have to do an explicit concatenation such as `"Hello " + n + " bottles of beer"`.

Functional-style Scala

But what can Scala offer after all our talk about functional-style programming throughout this book? The preceding code can be written in a more functional-style form in Java as follows:


```
                    main(String[] args) {
        IntStream.rangeClosed(2, 6)
                 .forEach(n -> System.out.println("Hello " + n +
                                                  " bottles of beer"));
```

```
    }
}
```

Here's how that code looks in Scala:

```
object Beer {
  def main(args: Array[String]){
    2 to 6 foreach { n => println(s"Hello ${n} bottles of beer") }
  }
}
```

The Scala code is similar to the Java code but less verbose. First, you can create a range by using the expression `2 to 6`. Here's something cool: `2` is an object of type `Int`. In Scala, *everything* is an object; there's no concept of primitive types, as in Java, which makes Scala a complete object-oriented language. An `Int` object in Scala supports a method named `to`, which takes as an argument another `Int` and returns a range. You could have written `2.to(6)` instead. But methods that take one argument can be written in an infix form. Next, `foreach` (with a lowercase `e`) is similar to `forEach` in Java (with an uppercase `E`). This method is available on a range (you use the infix notation again), and it takes a lambda expression as an argument to apply on each element. The lambda-expression syntax is similar to that in Java, but the arrow is => instead of –>.[1] The preceding code is functional; you're not mutating a variable as you did in the earlier example using a `while` loop.

1

Note that the Scala terms anonymous functions and closures (interchangeable) refer to what Java calls lambda expressions.

20.1.2. Basic data structures: List, Set, Map, Tuple, Stream, Option

Are you feeling good after having a couple of beers to quench your thirst? Most real programs need to manipulate and store data, so in this section, you manipulate collections in Scala and see how that process compares with Java.

Creating collections

mple, thanks to Scala's emphasis on con-

w to create a `Map`:

```
"Raoul" -> 23, "Mario" -> 40, "Alan" -> 53)
```

Several things are new in this line of code. First, it's awesome that you can create a `Map` and associate a key with a value directly, using the syn-

tax –>. There's no need to add elements manually, as in Java:

```
Map<String, Integer> authorsToAge = new HashMap<>();
authorsToAge.put("Raoul", 23);
authorsToAge.put("Mario", 40);
authorsToAge.put("Alan", 53);
```

You learned in **chapter 8**, however, that Java 9 has a couple of factory methods, inspired by Scala, that can help you tidy this type of code:

```
Map<String, Integer> authorsToAge
    = Map.ofEntries(entry("Raoul", 23),
                    entry("Mario", 40),
                    entry("Alan", 53));
```

The second new thing is that you can choose not to annotate the type of the variable `authorsToAge`. You could have explicitly written `val authorsToAge : Map[String, Int]`, but Scala can infer the type of the variable for you. (Note that the code is still checked statically. All variables have a given type at compile time.) We come back to this feature in **chapter 21**. Third, you use the `val` keyword instead of `var`. What's the difference? The keyword `val` means that the variable is read-only and can't be reassigned to (like `final` in Java). The `var` keyword means that the variable is read-write.

What about other collections? You can create a `List` (a singly linked list) or a `Set` (no duplicates) easily, as follows:

```
val authors = List("Raoul", "Mario", "Alan")
val numbers = Set(1, 1, 2, 3, 5, 8)
```

The `authors` variable has three elements, and the `numbers` variable has five elements.

Immutable vs. mutable

One important fact to keep in mind is that the collections you created previously are immutable by default, which means that they can't be changed after they're created. Immutability is useful because you know ny point in your program always yields a ts.

How can you update an immutable collection in Scala? To come back to the terminology used in **chapter 19**, such collections in Scala are said to be *persistent*. Updating a collection produces a new collection that shares as much as possible with the previous version, which persists without be-

ing affected by changes (as we show in figures 19.3 and 19.4). As a consequence of this property, your code has fewer implicit data dependencies: there's less confusion about which location in your code updates a collection (or any other shared data structure) and at what point in time.

The following example demonstrates this idea. Add an element to a `Set`:

```
val numbers = Set(2, 5, 3);
val newNumbers = numbers + 8        1
println(newNumbers)                 2
println(numbers)                    3
```

- ***1* Here, `+` is a method that adds 8 to the Set, creating a new Set object as a result.**
- ***2* (2, 5, 3, 8)**
- ***3* (2, 5, 3)**

In this example, the set of numbers isn't modified. Instead, a new `Set` is created with an additional element.

Note that Scala doesn't force you to use immutable collections—only makes it easy to adopt immutability in your code. Also, mutable versions are available in the package `scala.collection.mutable`.

Unmodifiable vs. immutable

Java provides several ways to create unmodifiable collections. In the following code, the variable `newNumbers` is a read-only view of the set `numbers`:

```
Set<Integer> numbers = new HashSet<>();
Set<Integer> newNumbers = Collections.unmodifiableSet(numbers);
```

This code means that you won't be able to add new elements through the `newNumbers` variable. But an unmodifiable collection is a wrapper over a modifiable collection, so you could still add elements by accessing the `numbers` variable.

By contrast, immutable collections guarantee that nothing can change the ny variables are pointing to it.

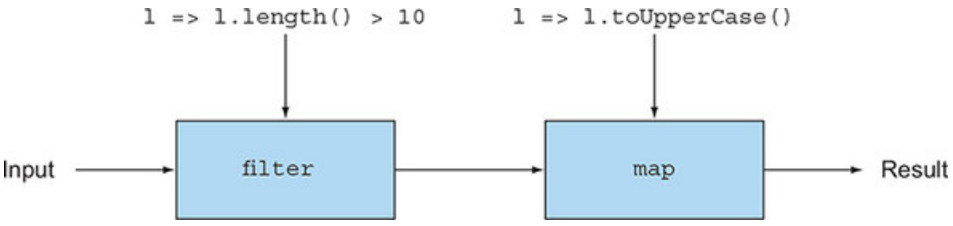
you could create a persistent data structure: an immutable data structure that preserves the previous version of itself when modified. Any modifications always produce a new updated structure.

Working with collections

Now that you've seen how to create collections, you need to know what you can do with them. Collections in Scala support operations similar to those in the Java `Stream` API. You may recognize `filter` and `map` in the following example and as illustrated in **figure 20.1**:

```
val fileLines = Source.fromFile("data.txt").getLines.toList()
val linesLongUpper
  = fileLines.filter(l => l.length() > 10)
             .map(l => l.toUpperCase())
```

Figure 20.1. Streamlike operations with Scala's `List`



Don't worry about the first line, which transforms a file into a list of strings consisting of the lines in the file (similar to what `Files.readAllLines` provides in Java). The second line creates a pipeline of two operations:

- A `filter` operation that selects only the lines that have a length greater than 10
- A `map` operation that transforms these long lines to uppercase

This code can be also written as follows:

```
val linesLongUpper
  = fileLines filter (_.length() > 10) map(_.toUpperCase())
```

You use the infix notation as well as the underscore (`_`) character, which is a placeholder that's positionally matched with any arguments. In this case, you can read `_.length()` as `l => l.length()`. In the functions passed to `filter` and `map`, the underscore is bound to the line parameter

e available in Scala's collection API. We recommend taking a look at the Scala documentation to get an idea (**https://docs.scala-lang.org/overviews/collections/introduction.html**). Note that this API is slightly richer than the Streams API (including support for zipping operations, which let you combine elements of two lists),

so you'll definitely gain a few programming idioms by checking it out. These idioms may also make it into the Streams API in future versions of Java.

Finally, remember that in Java, you can ask for a pipeline to be executed in parallel by calling `parallel` on a `Stream`. Scala has a similar trick. You need only use the `par` method:

```
val linesLongUpper
  = fileLines.par filter (_.length() > 10) map(_.toUpperCase())
```

Tuples

This section looks at another feature that's often painfully verbose in Java: tuples. You may want to use tuples to group people by name and phone number (here, simple pairs) without declaring an ad hoc new class and instantiating an object for it: `("Raoul", "+44 7700 700042")`, `("Alan", "+44 7700 700314")`, and so on.

Unfortunately, Java doesn't provide support for tuples, so you have to create your own data structure. Here's a simple `Pair` class:

```
public class Pair<X, Y> {
    public final X x;
    public final Y y;
    public Pair(X x, Y y){
        this.x = x;
        this.y = y;
    }
}
```

Also, of course you need to instantiate pairs explicitly:

```
Pair<String, String> raoul = new Pair<>("Raoul", "+44 7700 700042");
Pair<String, String> alan = new Pair<>("Alan", "+44 7700 700314");
```

Okay, but how about triplets and arbitrary-size tuples? Defining a new class for every tuple size is tedious and ultimately affects the readability and maintainability of your programs.

ch allow you to create tuples through
ormal mathematical notation, as follows:

```
val raoul = ("Raoul", "+44 7700 700042")
val alan = ("Alan", "+44 7700 700314")
```

Scala supports arbitrary-size[2] tuples, so the following are possible:

2

Tuples have a limit of 22 elements.

```
val book = (2018 "Modern Java in Action", "Manning")      1
val numbers = (42, 1337, 0, 3, 14)                        2
```

- ***1*** **A tuple of type (Int, String, String)**
- ***2*** **A tuple of type (Int, Int, Int, Int, Int)**

You can access the elements of the tuples by their positions by using the accessors _1, _2 (starting at 1), as in this example:

```
println(book._1)          1
println(numbers._4)       2
```

- ***1*** **Prints 2018**
- ***2*** **Prints 3**

Isn't that example much nicer than what you'd have to write in Java? The good news is that there are discussions about introducing tuple literals in future versions of Java. (See **chapter 21** for more discussion of possible new features in Java.)

Stream

The collections that we've described so far—`List`, `Set`, `Map`, and `Tuple`—are all evaluated eagerly (that is, immediately). By now, you know that streams in Java are evaluated on demand (that is, lazily). You saw in **chapter 5** that because of this property, streams can represent an infinite sequence without overflowing memory.

Scala also provides a corresponding lazily evaluated data structure called `Stream`. But `Streams` in Scala provide more features than those in Java. `Streams` in Scala remember values that were computed so that previous elements can be accessed. In addition, `Streams` are indexed so that elements can be accessed by an index, like a list. Note that the trade-off for these additional properties is the fact that `Streams` are less memory-efficient than `Streams`, because being able to refer to previous elements need to be remembered (cached).

Option

Another data structure that you'll be familiar with is `Option`—Scala's version of Java's `Optional`, which we discussed in **chapter 11**. We argued

that you should use `Optional` when possible to design better APIs, in which by reading the signature of a method, users can tell whether they can expect an optional value. You should use this data structure instead of `null` whenever possible to prevent `null`-pointer exceptions.

You saw in **chapter 11** that you can use `Optional` to return the name of a person's insurance if the person's age is greater than some minimum age, as follows:

```
public String getCarInsuranceName(Optional<Person> person, int minAge) {
    return person.filter(p -> p.getAge() >= minAge)
                 .flatMap(Person::getCar)
                 .flatMap(Car::getInsurance)
                 .map(Insurance::getName)
                 .orElse("Unknown");
}
```

In Scala, you can use `Option` in a way similar to `Optional`:

```
def getCarInsuranceName(person: Option[Person], minAge: Int) =
  person.filter(_.age >= minAge)
        .flatMap(_.car)
        .flatMap(_.insurance)
        .map(_.name)
        .getOrElse("Unknown")
```

You can recognize the same structure and method names apart from `getOrElse`, which is the equivalent of `orElse` in Java. You see, throughout this book, you've learned new concepts that you can apply directly to other programming languages! Unfortunately, `null` also exists in Scala for Java compatibility reasons, but its use is highly discouraged.

20.2. Functions

Scala functions can be viewed as being sequences of instructions that are grouped to perform a task. These functions are useful for abstracting behavior and are the cornerstone of functional programming.

In Java, you're familiar with *methods*: functions associated with a class. You've also seen lambda expressions, which can be considered to be rs a richer set of features to support func- k at those features in this section. Scala

- *Function types*—syntactic sugar that represents the idea of Java function descriptors (that is, notations that represent the signature of the abstract method declared in a functional interface), which we described in **chapter 3**

- Anonymous functions that don't have the no-writing restriction on nonlocal variables that Java's lambda expressions have
- Support for *currying*, which means breaking a function that takes multiple arguments into a series of functions, each of which takes some of the arguments

20.2.1. First-class functions in Scala

Functions in Scala are *first-class values*, which means that they can be passed around as parameters, returned as a result, and stored in variables, like values such as `Integer` and `String`. As we've shown you in earlier chapters, method references and lambda expressions in Java can also be seen as first-class functions.

Here's an example of how first-class functions work in Scala. Suppose that you have a list of strings representing tweets you've received. You'd like to filter this list with different criteria, such as tweets that mention the word *Java* or tweets of a certain short length. You can represent these two criteria as *predicates* (functions that return a `Boolean`):

```
def isJavaMentioned(tweet: String) : Boolean = tweet.contains("Java")
def isShortTweet(tweet: String) : Boolean = tweet.length() < 20
```

In Scala, you can pass these methods directly to the built-in `filter` as follows (as you can pass them by using method references in Java):

```
val tweets = List(
    "I love the new features in Java",
    "How's it going?",
    "An SQL query walks into a bar, sees two tables and says 'Can I join
)
tweets.filter(isJavaMentioned).foreach(println)
tweets.filter(isShortTweet).foreach(println)
```

Now inspect the signature of the built-in method `filter`:

```
def filter[T](p: (T) => Boolean): List[T]
```

You may wonder what the type of the parameter `p` means (here, `(T) =>` l expect a functional interface. This Scala a, but it describes a *function type*. Here, at takes an object of type `T` and returns a `Boolean`. In Java, this type is expressed as a `Predicate<T>` or `Function<T, Boolean>`, which has the same signature as the `isJavaMentioned` and `isShortTweet` methods, so you can pass them as arguments to `filter`. The designers of the Java language decided not to intro-

duce similar syntax for function types to keep the language consistent with previous versions. (Introducing too much new syntax in a new version of the language is viewed as adding too much cognitive overhead.)

20.2.2. Anonymous functions and closures

Scala also supports anonymous functions, which have syntax similar to that of lambda expressions. In the following example, you can assign to a variable named `isLong–Tweet` an anonymous function that checks whether a given tweet is long:

```
val isLongTweet : String => Boolean                 1
    = (tweet : String) => tweet.length() > 60       2
```

- ***1* A variable of function type String to Boolean**
- ***2* An anonymous function**

In Java, a lambda expression lets you create an instance of a functional interface. Scala has a similar mechanism. The preceding code is syntactic sugar for declaring an anonymous class of type `scala.Function1` (a function of one parameter), which provides the implementation of the method `apply`:

```
val isLongTweet : String => Boolean
    = new Function1[String, Boolean] {
        def apply(tweet: String): Boolean = tweet.length() > 60
      }
```

Because the variable `isLongTweet` holds an object of type `Function1`, you can call the method `apply`, which can be seen as calling the function:

```
isLongTweet.apply("A very short tweet")           1
```

- ***1* Returns false**

In Java, you could do the following:

```
Function<String, Boolean> isLongTweet = (String s) -> s.length() > 60;
boolean long = isLongTweet.apply("A very short tweet");
```


essions, Java provides several built-in functional interfaces such as `Predicate`, `Function`, and `Consumer`. Scala provides traits (you can think of traits as being interfaces for now) to achieve the same thing: `Function0` (a function with 0 parameters and a

return result) up to `Function22` (a function with 22 parameters), all of which define the method `apply`.

Another cool trick in Scala allows you to implicitly call the `apply` method by using syntactic sugar that looks more like a function call:

```
isLongTweet("A very short tweet")          1
```

- ***1*** **Returns false**

The compiler automatically converts a call `f(a)` to `f.apply(a)` and, more generally, a call `f(a1, ..., an)` to `f.apply(a1, ..., an)`, if `f` is an object that supports the `apply` method. (Note that `apply` can have any number of arguments.)

Closures

In **chapter 3**, we commented on whether lambda expressions in Java constitute closures. A *closure* is an instance of a function that can reference nonlocal variables of that function with no restrictions. But lambda expressions in Java have a restriction: they can't modify the content of local variables of a method in which the lambda is defined. Those variables have to be implicitly `final`. It helps to think that lambdas close over values, rather than variables.

By contrast, anonymous functions in Scala can capture variables themselves, not the *values* to which the variables currently refer. The following is possible in Scala:

```
def main(args: Array[String]) {
    var count = 0
    val inc = () => count+=1          1
    inc()
    println(count)                    2
    inc()
    println(count)                    3
}
```

- ***1*** **A closure capturing and incrementing count**
- ***2*** **Prints 1**

in a compiler error because `count` is implicitly forced to be `final`.

```
public static void main(String[] args) {
    int count = 0;
```

```
    Runnable inc = () -> count+=1;              1
    inc.run();
    System.out.println(count);
    inc.run();
}
```

- ***1*** **Error: Count must be final or effectively final.**

We argued in **chapters 7**, **18**, and **19** that you should avoid mutation whenever possible to make your programs easier to maintain and parallelizable, so use this feature only when strictly necessary.

20.2.3. Currying

In **chapter 19**, we described a technique called *currying*, in which a function `f` of two arguments (`x` and `y`, say) is seen instead as a function `g` of one argument, which returns a function also of one argument. This definition can be generalized to functions with multiple arguments, producing multiple functions of one argument. In other words, you can break down a function that takes multiple arguments into a series of functions, each of which takes a subset of the arguments. Scala provides a construct that makes it easy to curry an existing function.

To understand what Scala brings to the table, first revisit an example in Java. You can define a simple method to multiply two integers:

```
static int multiply(int x, int y) {
  return x * y;
}
int r = multiply(2, 10);
```

But this definition requires all the arguments to be passed to it. You can break down the `multiply` method manually by making it return another function:

```
static Function<Integer, Integer> multiplyCurry(int x) {
    return (Integer y) -> x * y;
}
```

The function returned by `multiplyCurry` captures the value of `x` and eturning an `Integer`. As a result, you can n a `map` to multiply each element by 2:


```
Stream.of(1, 3, 5, 7)
      .map(multiplyCurry(2))
      .forEach(System.out::println);
```

This code produces the result 2, 6, 10, 14. This code works because `map` expects a `Function` as argument and `multiplyCurry` returns a `Function`.

It's a bit tedious in Java to split up a function manually to create a curried form, especially if the function has multiple arguments. Scala has a special syntax that performs this automatically. You can define the normal `multiply` method as follows:

```
def multiply(x : Int, y: Int) = x * y
val r = multiply(2, 10)
```

And here is the curried form:

```
def multiplyCurry(x :Int)(y : Int) = x * y      1
val r = multiplyCurry(2)(10)                    2
```

- ***1*** **Defining a curried function**
- ***2*** **Invoking a curried function**

When you use the `(x: Int)(y: Int)` syntax, the `multiplyCurry` method takes *two* argument lists of one `Int` parameter. By contrast, `multiply` takes *one list* of two `Int` parameters. What happens when you call `multiplyCurry`? The first invocation of `multiplyCurry` with a single `Int` (the parameter `x`), `multiplyCurry(2)`, returns another function that takes a parameter `y` and multiplies it by the captured value of `x` (here, the value 2). We say that this function is *partially applied* as explained in **chapter 19**, because not all arguments are provided. The second invocation multiplies `x` and `y`. You can store the first invocation to `multiplyCurry` inside a variable and reuse it, as follows:

```
val multiplyByTwo : Int => Int = multiplyCurry(2)
val r = multiplyByTwo(10)                          1
```

- ***1*** **20**

By comparison with Java, in Scala you don't need to provide the curried form of a function manually, as in the preceding example. Scala provides a convenient function-definition syntax to indicate that a function has

In this section, we look at how classes and interfaces in Java compare with those in Scala. These two constructs are paramount to design applications. You'll see that Scala's classes and interfaces can provide more flexibility than those in Java.

20.3.1. Less verbosity with Scala classes

Because Scala is a full object-oriented language, you can create classes and instantiate them to produce objects. In its most basic form, the syntax to declare and instantiate classes is similar to that of Java. Here's how to declare a `Hello` class:

```
class Hello {
  def sayThankYou(){
    println("Thanks for reading our book")
  }
}
val h = new Hello()
h.sayThankYou()
```

Getters and setters

Scala becomes more interesting when you have a class with fields. Have you ever come across a Java class that purely defines a list of fields and had to declare a long list of getters, setters, and an appropriate constructor? What a pain! In addition, you often see tests for the implementation of each method. A large amount of code typically is devoted to such classes in Enterprise Java applications. Consider this simple `Student` class:

```
public class Student {
    private String name;
    private int id;
    public Student(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
}
```


constructor that initializes all fields, two getters, and two setters. A simple class now has more than 20 lines of code. Several IDEs (integrated development environments) and tools can help you generate this code, but your code base still has to deal with a

large amount of additional code that's not too useful compared with real business logic.

In Scala, constructors, getters, and setters can be generated implicitly, which results in code with less verbosity:

```
class Student(var name: String, var id: Int)
val s = new Student("Raoul", 1)                  1
println(s.name)                                  2
s.id = 1337                                      3
println(s.id)                                    4
```

- ***1*** **Initialize a Student object.**
- ***2*** **Get the name and print Raoul.**
- ***3*** **Set the id.**
- ***4*** **Print 1337.**

In Java, you could get similar behavior by defining public fields, but you'd still have to define the constructor explicitly. Scala classes save you boilerplate code.

20.3.2. Scala traits vs. Java interfaces

Scala has another useful feature for abstraction, called traits, which are Scala's replacement for Java's interfaces. A trait can define both abstract methods and methods with a default implementation. Traits can also be multiply inherited like interfaces in Java, so you can see them as being similar to Java interfaces that support default methods. Traits can also contain fields such as abstract classes, which Java interfaces don't support. Are traits like abstract classes? No, because unlike abstract classes, traits can be multiply inherited. Java has always had multiple inheritance of types because a class can implement multiple interfaces. Java 8, through default methods, introduced multiple inheritance of behaviors but still doesn't allow multiple inheritance of state—something permitted by Scala traits.

To see what a trait looks like in Scala, define a trait called `Sized` that contains one mutable field called `size` and one method called `isEmpty` with a default implementation:


```
                                                 1
== 0                                             2
```

- ***1*** **A field called size**
- ***2*** **A method called isEmpty with a default implementation**

You can compose this code at declaration time with a class, such as an `Empty` class that always has size 0:

```
class Empty extends Sized          1
println(new Empty().isEmpty())     2
```

- ***1* A class inheriting from the trait Sized**
- ***2* Prints true**

Interestingly, compared with Java interfaces, traits can be composed at object instantiation time (but this operation is still a compile-time operation). You can create a `Box` class and decide that one specific instance should support the operations defined by the trait `Sized`, as follows:

```
class Box
val b1 = new Box() with Sized      1
println(b1.isEmpty())              2
val b2 = new Box()
b2.isEmpty()                       3
```

- ***1* Composing the trait at object instantiation time**
- ***2* Prints true**
- ***3* Compile error: The Box class declaration doesn't inherit from Sized.**

What happens if multiple traits are inherited, declaring methods with the same signatures or fields with the same names? Scala provides restriction rules similar to those that apply to default methods (**chapter 13**).

Summary

- Java and Scala combine object-oriented and functional-programming features into one programming language; both run on the JVM and to a large extent can interoperate.
- Scala supports collection abstractions similar to those in Java—`List`, `Set`, `Map`, `Stream`, `Option`—but also supports tuples.
- Scala provides richer features that support more functions than Java does. These features include function types, closures that have no restrictions on accessing local variables, and built-in currying forms.
- [illegible]mplicit constructors, getters, and setters.
- [illegible]re interfaces that can include fields and